Aayush Mediratta

Orlando, FL · live.yush@gmail.com · mercpl.us · LinkedIn · GitHub

Northstar: A Multi-Agent Market Intelligence Workspace

Every founder deserves an analyst team. Northstar gives you one that researches live, audits itself, and then argues with you.

TRACK: AGENTS FOR BUSINESS

The Problem

Before anyone writes a line of product code, they need answers to three questions. Who else is doing this? How big is the market? Will anyone actually pay for it?

Today the honest answer for most founders is a weekend of chaos. Twenty browser tabs, a half-finished spreadsheet, a TAM number copied from a random blog post, and a gut feeling. The research is stale the moment it is done, the sources are lost, and there is no way to tell which numbers came from a real citation and which ones were invented to make the slide look complete.

I have lived this. Every time I evaluate a startup idea, I burn days redoing the same manual loop: search, skim, copy, guess, repeat. Investors see the output and immediately ask the question that kills the meeting: "where did this number come from?"

That question is the whole problem. Market research is not hard because the information does not exist. It is hard because collecting it, verifying it, and keeping facts separate from assumptions is a multi-step, multi-role workflow. Which makes it a perfect job for a multi-agent system.

The Solution

Northstar is a market intelligence workspace that takes one startup concept and produces a complete, audited market brief in a few minutes. It runs live web research through the Parallel Search API, sizes the market bottom-up and top-down with real citations, and writes everything into a single portable artifact, a hybrid Markdown plus YAML file called `[project].market.md`.

Then it goes further than a report. The same artifact powers three interactive modes:

- **Advisory Board**: pitch your pricing or features to AI customer personas generated from the research. Each persona is a constrained agent with its own demographics, budget, and buying triggers. They score fit, raise friction points, and give a buy or pass decision.
- **War Room**: inject a launch scenario like "launching at $49/mo with offline support" and watch competitor executive agents, each locked to a real competitor profile scraped from the web, respond with pricing reactions, feature counters, and a threat matrix.
- **Landing Page**: the differentiation angle, value hooks, and objection handling copy synthesized from competitor weaknesses and persona pushback, ready to test.

The one design rule that runs through everything: **facts and assumptions never mix**. Every number in Northstar is labeled either as a verified extraction with a source URL, or as a modeled assumption with its reasoning attached. The UI renders them in separate panels. The artifact stores them in separate provenance

sections. When a price is estimated instead of extracted, the interface says "~$65/mo est." instead of pretending it is real.

Why Agents

A single LLM call cannot do this job well, and I know because my first version tried. This workflow needs different roles with different responsibilities that check each other:

1. Planning needs world knowledge (what category is this concept, who are the real competitors).
2. Research needs live tools (the web changes daily, training data does not).
3. Auditing needs to be adversarial to research (a researcher grading its own homework passes everything).
4. Simulation needs persistent, constrained personas (a buyer agent that breaks character is useless).

Northstar maps each role to an agent and wires them into a pipeline with an explicit feedback loop. The auditor can and does fail the researcher, which triggers a revision cycle. You can watch this happen live in the Agent Log.

Architecture

```
Frontend (SPA, SSE streaming)
        |
FastAPI (app/main.py)
        |
Scout Orchestrator (app/scout/orchestrator.py)
        |
  +-----+-------------------------------------------+
  |                                                 |
Research pipeline                          Simulation workflows (Google ADK)
  |                                                 |
  1. Research Planner (Gemini 3.1 Pro)      Advisory Board: ParallelAgent of
     category, seed competitors,            persona agents + SequentialAgent
     search queries, buyer estimates,       synthesizer
     personas, positioning                  War Room: ParallelAgent of
  2. Live Search (Parallel Search API,      competitor executive agents +
     advanced mode) + Extract               synthesizer
  3. Competitor Auditor (Gemini)            Both built dynamically from agent
     rejects blogs, docs, non-competitors   blueprints stored in the artifact
  4. TAM/SAM/SOM engine with top-down
     reconciliation
  5. Auditor Agent: schema, URLs,
     numeric sanity. Fail = retry loop
        |
Agent Skills (.agent/skills/scout/)        MCP Server (FastMCP at /mcp)
  SKILL.md + market_schema.yaml              search_competitors
  verify_links.py                            search_market_size
  fetch_tam_formula.py                       artifact load/save with a
  pricing_normalizer.py                      workspace path guard
  evidence_normalizer.py
  artifact_qa.py
```

The flow for one research run:

Plan. A Gemini 3.1 Pro planner agent turns the raw concept into a research plan: the market category, 4 to 8 real competitor names to seed discovery, concept-specific search queries, market-size search terms, buyer count estimates with rationale, two buyer personas, and a positioning wedge. This one step was the single biggest quality unlock in the project (more on that in the journey section).

Search. All web access goes through the Parallel Search API in advanced mode. Discovery queries plus planner-seeded vendor lookups return candidate pages, then Parallel Extract pulls structured excerpts from

the winners. There is no scraping of search engine result pages in the live path.

Validate. A second Gemini agent, the competitor auditor, reviews every scraped candidate against the concept. It rejects blog posts about competitors, documentation pages, app store listings, and brands that do not actually compete. It also fixes mislabeled names (a LangChain repo is LangChain, not "GitHub") and writes a concept-specific weakness for each kept competitor.

Size. The TAM engine computes bottom-up sizing from extracted competitor prices and modeled buyer counts, then searches for a third-party top-down citation and reconciles the two. If bottom-up TAM exceeds the cited total market, the buyer count is capped and a plain-English reconciliation note is written into the artifact. Your addressable market cannot be bigger than the market.

Audit. A separate auditor agent validates the draft against the skill's YAML schema, checks that every competitor URL actually resolves, verifies all numeric fields are finite, and enforces that top-down and bottom-up TAM agree within 10x. On failure, the researcher revises and the loop runs again, up to two retries.

Write. The artifact is rendered as hybrid Markdown with a structured YAML block, saved to disk, previewable in the UI, and exportable as PDF. The Advisory Board and War Room load their agent blueprints directly from it, so the simulations are always grounded in the same evidence the report shows.

Course Concepts Demonstrated

1. Multi-agent system with Google ADK (code). The Advisory Board and War Room are real ADK workflows: a `ParallelAgent` fans out persona or competitor executive agents (each with a JIT system instruction built from blueprints in the artifact), and a `SequentialAgent` runs a synthesizer that merges their JSON outputs into fit scores, threat levels, and risk matrices. Sessions persist through ADK's `InMemorySessionService`, so multi-turn follow-ups keep context. The research pipeline itself is planner, researcher, competitor auditor, and document auditor coordinated by an orchestrator with a fail-and-revise loop.

2. MCP Server (code). Northstar mounts a FastMCP server at `/mcp` exposing `search_competitors`, `search_market_size`, and artifact read/write tools. The research pipeline is also its own MCP client: the TAM engine calls `search_market_size` through the MCP interface first, with a direct API fallback. Any MCP-compatible agent can drive Northstar's research tools externally.

3. Agent Skills (code). The research behavior lives in `.agent/skills/scout/` as a proper skill: a SKILL.md contract describing when to use it, a YAML schema the auditor validates against, and five executable scripts (link verification, TAM formulas, pricing normalization, evidence normalization, artifact QA) that the runtime loads dynamically. The pipeline logic and the skill definition stay in sync because the pipeline literally executes the skill's scripts.

4. Security features (code). No keys in code, everything through environment variables with `.env` gitignored. The MCP artifact tools enforce a workspace path guard that rejects any path resolving outside the project root. All user and web-sourced strings are HTML-escaped before rendering. The evidence discipline itself is a safety feature: the system is structurally incapable of presenting a modeled number as a cited fact.

5. Deployability (code and video). One command, `./restart.sh`, builds and starts the full stack via Docker Compose, waits on the `/health` endpoint, and serves the app at localhost:8000. The health endpoint reports backend readiness for every subsystem so you can diagnose a misconfigured key in seconds.

The Journey: What Actually Went Wrong

The first working version looked great and was quietly wrong, and I think the story of fixing it is the most valuable part of this project.

I tested it with a description of a well-known AI agent integration platform. It confidently returned six competitors: GitHub, DeepAI, Google AI, "OpenRouter Documentation," and a data catalog company whose blog post about LangChain got attributed as their own product. Not one real competitor. The prices were keyword-bucket guesses stamped with a "Public-Price" badge. The TAM used a generic "General B2B operators" profile regardless of concept.

The root cause: the pipeline was pure heuristics. Regex name extraction, keyword category matching, and a fallback that scraped search engine results when the primary source looked thin. No step in the system ever asked the question a human analyst asks first: is this actually a competitor?

The fix reshaped the architecture around a principle I would give any agent builder: **use the LLM for judgment, use tools for facts, and never let a heuristic make a semantic decision.**

- The planner agent now front-loads judgment: real category, real competitor names, grounded buyer estimates.
- The competitor auditor back-stops with judgment: every candidate gets an explicit keep-or-reject verdict with a reason, and rejections are logged transparently in the trace ("rejected: this is an App Store listing, the product is BallparkDeal").
- Parallel Search became the only search provider in the live path, replacing scraped result pages that anti-bot walls kept degrading.
- Honesty became enforceable: prices are "verified" only when a number was actually parsed from a pricing page, otherwise they are labeled as estimates and anchored to the planner's category price band.
- The TAM reconciliation rule was born from a real failure where bottom-up TAM came out 3x larger than the entire cited market.

After the rework, the same test methodology on a real estate investment analysis concept returned Mashvisor, BiggerPockets, and PropStream, which are exactly the right competitors, with a cited $5.24B top-down market from a named research report, personas specific enough to have names and salaries, and an audit trail for every claim.

Value

Northstar compresses a multi-day research task into minutes, but speed is not the point. The point is trust. Every artifact it produces can survive the "where did this number come from" question, because the answer is attached to the number. For a founder, that is the difference between a deck and a defensible plan. For a judge, every claim in the demo is one click from its source.

The artifact format is the quiet superpower. Because research, simulation state, and positioning all live in one schema-validated file, the whole workspace is portable, diffable, and reusable by other agents through MCP. Run research once, then interrogate it from four different angles without the story ever drifting from the evidence.

The Build

FastAPI backend, vanilla JS frontend with SSE streaming for the live agent trace, Google ADK with Gemini 3.1 Pro Preview for all agent reasoning, Parallel Search and Extract APIs for all web access, FastMCP for the tool server, Docker Compose for deployment. The full agent activity log streams to the UI during a run, so you can

watch the planner, searcher, auditor, and reconciliation steps happen in real time, including the failures and retries.

Links

- Live demo for testing: https://aayush.foo
- Video demo: attached in the Media Gallery
- Public code repository with setup instructions: attached project link
- Run it yourself: clone, add `GOOGLE_API_KEY` and `PARALLEL_API_KEY` to `.env`, then `./restart.sh` and open localhost:8000